



CASE STUDY DOCUMENT

# Parking Lot System

## Smart Parking Management Platform

### ***ParkEase***

*Find. Reserve. Park. Effortlessly.*

Version 1.0 | 2026



# 1. Synopsis

ParkEase is a full-stack Smart Parking Lot Management System that connects drivers seeking parking with lot operators offering managed parking spaces. The platform supports three primary roles: Drivers (vehicle owners seeking parking), Lot Managers (parking facility operators), and Platform Administrators — each with a role-specific dashboard and a distinct set of capabilities.

Drivers can discover nearby parking lots by location or city, browse real-time spot availability, filter by vehicle type or spot features (EV charging, handicapped access), reserve a specific spot in advance or on arrival, check in and check out digitally, and pay online. Lot Managers can register and configure parking lots, define spot layouts with per-spot pricing and type classification, manage bookings, and access occupancy and revenue analytics. Administrators oversee the entire ecosystem: approving new lots, managing users, viewing platform-wide analytics, and generating financial reports.

ParkEase is architected as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, parking lot profiles, parking spot management, booking/reservation lifecycle, payment processing, vehicle management, notifications, and analytics/reporting — all integrated through a Spring MVC website controller layer.

## 2. Detailed Requirements

### 2.1 General Requirements

- Guests can search and browse parking lots and view spot availability without logging in.
- Users must register or log in (email/password or OAuth) to create bookings or manage lots.
- Role-based access control: Driver, Lot Manager, and Admin with distinct permission sets.
- All users can update their profile, contact information, and password.
- In-app, email, and SMS notifications are dispatched for all key booking and payment events.
- Admin panel provides full platform management, lot approval, user management, and analytics.

## 2.2 Driver Requirements

- Register and log in via email/password or Google/GitHub OAuth.
- Register one or more vehicles with license plate, make, model, vehicle type (2W/4W/Heavy), and EV flag.
- Search for parking lots by city, address, or landmark; view nearby lots using GPS-based proximity.
- Filter lots by available spot types (Compact, Standard, Large, EV-only, Handicapped-accessible).
- View real-time spot availability for a selected lot with floor-level breakdown.
- Reserve an available spot in advance (pre-booking) or on arrival (walk-in booking).
- Cancel a booking within the allowed cancellation window; receive a refund if applicable.

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

- Extend the parking duration if the spot remains available.
- Check in digitally on arrival (updates spot status to Occupied).
- Check out on departure (triggers fare calculation and payment processing).
- Pay parking fees online (wallet, card, UPI) or opt for pay-on-exit (cash or card at gate).
- View complete booking history and download payment receipts.
- Receive notifications for: booking confirmation, check-in alert, expiry reminder, checkout confirmation, payment receipt.

## 2.3 Lot Manager Requirements

- Register a parking lot with name, address, GPS coordinates, operating hours, and images.
- Await admin approval before the lot becomes publicly visible and bookable.
- Add individual spots or bulk-create spots with spot number, floor, type, vehicle type compatibility, price per hour, EV-charging flag, and handicapped flag.
- Edit spot details including per-spot pricing and feature flags.
- Toggle the lot's open/closed status in real time (e.g., for maintenance).
- View all incoming bookings and active check-ins for the lot.
- Manually update booking status where required (e.g., force checkout for vehicles exceeding duration).
- View occupancy analytics: current rate, trends by hour, peak hours, and spot type utilisation.
- View revenue analytics: earnings by day, week, or month; total revenue for any date range.

Generate downloadable daily occupancy and revenue reports.

- Receive notifications for: new booking at their lot, early checkout, extended stay.

## 2.4 Admin Requirements

- View and manage all user accounts: drivers and lot managers — suspend, reactivate, or permanently delete.
- Review new lot registration submissions and approve or reject with feedback.
- View all bookings platform-wide with full lifecycle status and associated payments.
- Access platform analytics: total bookings, total revenue, most booked cities, vehicle type breakdown.
- Generate platform-wide revenue reports for financial reconciliation.
- Send platform-wide broadcast notifications to drivers, managers, or all users.
- View audit logs of all critical actions (lot approval, user suspension, manual checkout).

## 2.5 Booking & Reservation Requirements

- A booking links a driver, a specific spot, and a time window (startTime to endTime).
- Booking types: Pre-Booking (advance reservation with deposit) and Walk-in (immediate, no advance payment required).

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

- Spot status transitions: Available → Reserved (on booking) → Occupied (on check-in) → Available (on checkout or cancellation).
- Fare is calculated as:  $(\text{checkOutTime} - \text{checkInTime}) \times \text{spot.pricePerHour}$  with a minimum 1-hour charge.
- Extended bookings recalculate fare for the additional duration at checkout.
- Expired pre-bookings (no check-in within grace period) are auto-cancelled by a scheduled job.

## 2.6 Payment Requirements

- Drivers can pay online at booking (pre-booking deposit) or at checkout (actual computed fare).
- Payment modes: card, UPI, wallet balance, or cash on exit (COD).
- Each payment record stores amount, mode, transaction ID, status, and timestamps.
- Refunds are processed for eligible cancellations and credited to the original payment mode or wallet.
- Receipts are generated as PDF documents accessible from the booking detail page.

## 2.7 Analytics Requirements

- Real-time occupancy rate (%) computed as  $(\text{occupied spots} / \text{total spots}) \times 100$ .
- Hourly occupancy breakdown shows average fill level for each hour of the day.
- Peak hours analysis identifies the busiest time windows for a lot.

- Revenue reports aggregate total earnings for any date range, broken down by day.
  - Spot type utilisation report shows which categories (Standard, Compact, EV, etc.) are most booked.
  - Average parking duration is computed across all completed bookings for a lot.
- Platform-level summary aggregates across all active lots for admin dashboard.

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

### 3. Use Case Diagram

The diagram below captures all actors and use cases within the ParkEase platform — spanning authentication, lot discovery, vehicle management, spot booking lifecycle, payment, lot management, notifications, analytics, and administrative oversight.

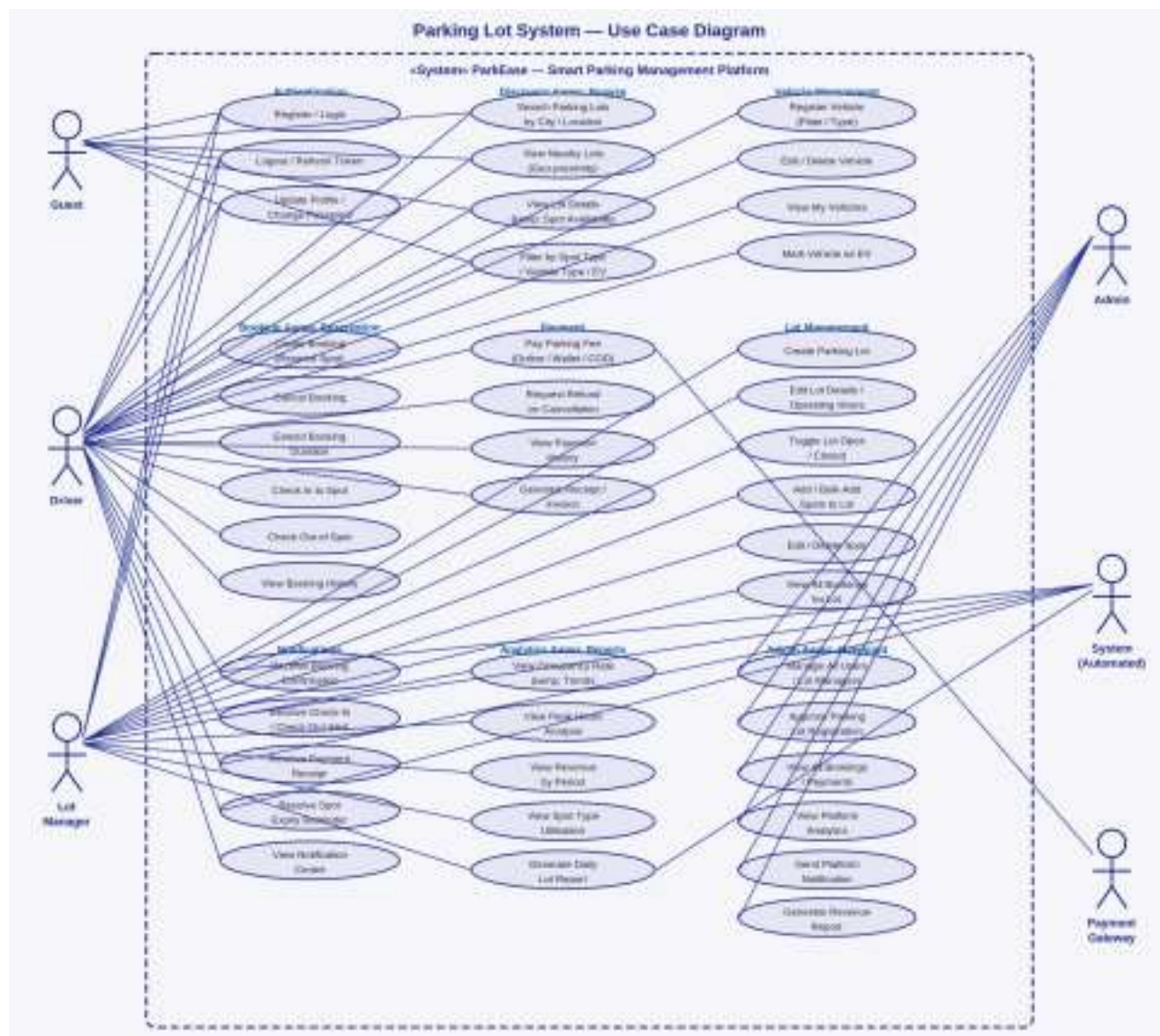


Figure 1: ParkEase Parking Lot System — Complete Use Case Diagram

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

### 3.1 Actors

|       |                                                                                |
|-------|--------------------------------------------------------------------------------|
|       |                                                                                |
| Guest | Unauthenticated visitor who can search parking lots and view spot availability |

|                    |                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------|
| Driver             | Registered vehicle owner who reserves spots, checks in/out, and makes payments                |
| Lot Manager        | Parking facility operator who registers lots, manages spots, and views analytics              |
| Admin              | Platform administrator who approves lots, manages users, and views platform-wide data         |
| System (Automated) | Automated component handling expired booking cancellations, reminders, and receipt generation |
| Payment Gateway    | External service processing card and UPI payment transactions                                 |

### 3.2 Use Case Summary

|                       |                        |                                                                        |
|-----------------------|------------------------|------------------------------------------------------------------------|
|                       |                        |                                                                        |
| Register / Login      | Guest, Driver, Manager | Create account or authenticate via email or OAuth                      |
| Update Profile        | Driver, Manager        | Change name, phone, profile picture, or password                       |
| Register Vehicle      | Driver                 | Add a vehicle with plate number, type, make, model, and EV flag        |
| Edit / Delete Vehicle | Driver                 | Modify or remove a registered vehicle from the account                 |
| View My Vehicles      | Driver                 | List all vehicles registered to the driver's account                   |
| Search Lots by City   | Guest, Driver          | Find parking lots in a specified city or area                          |
| View Nearby Lots      | Guest, Driver          | Discover lots within a configurable radius of current GPS location     |
| View Lot Details      | Guest, Driver          | See lot info, operating hours, available spot count, and pricing       |
| Filter by Spot Type   | Guest, Driver          | Narrow lot/spot results by vehicle type, EV charging, or accessibility |
| Create Booking        | Driver                 | Reserve a specific available spot for a time window                    |

Cancel Booking Driver Cancel a reservation; trigger refund if within cancellation policy

|                           |                         |                                                                  |
|---------------------------|-------------------------|------------------------------------------------------------------|
|                           |                         |                                                                  |
| Extend Booking            | Driver                  | Increase the parking duration if the spot remains available      |
| Check In to Spot          | Driver                  | Digitally confirm arrival; spot status changes to Occupied       |
| Check Out of Spot         | Driver                  | Confirm departure; fare is computed and payment triggered        |
| View Booking History      | Driver                  | Browse all past and upcoming bookings with status                |
| Make Payment              | Driver, Payment Gateway | Pay parking fee via card, UPI, wallet, or cash on exit           |
| Request Refund            | Driver                  | Submit refund for eligible cancelled or unused bookings          |
| View Payment History      | Driver                  | See all payment transactions and download receipts               |
| Generate Receipt          | Driver, System          | Download PDF payment receipt for a completed booking             |
| Create Parking Lot        | Lot Manager             | Register a new parking facility with address and operating hours |
| Edit Lot Details          | Lot Manager             | Modify lot info, operating hours, or geo-coordinates             |
| Toggle Lot Open/Closed    | Lot Manager             | Instantly open or close the lot to new bookings                  |
| Add / Bulk-Add Spots      | Lot Manager             | Create individual or batch parking spots with type and pricing   |
| Edit / Delete Spot        | Lot Manager             | Modify spot details (pricing, type, EV/handicapped flags)        |
| View All Bookings for Lot | Lot Manager             | See all active and historical bookings for managed lots          |
| View Occupancy Rate       | Lot Manager             | Monitor real-time fill percentage with hourly trends             |
| View Peak Hours Analysis  | Lot Manager             | Identify the busiest time windows to adjust pricing              |
| View Revenue by Period    | Lot Manager             | Track earnings over custom date ranges with daily breakdown      |
| Generate Daily Lot Report | Lot Manager             | Download occupancy and revenue summary for the day               |
| Approve Lot Registration  | Admin                   | Review and approve or reject a manager's lot submission          |
| Manage All Users          | Admin                   | View, suspend, reactivate, or delete any driver                  |



|  |  |            |
|--|--|------------|
|  |  | or manager |
|--|--|------------|

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

|                             |        |                                                             |
|-----------------------------|--------|-------------------------------------------------------------|
|                             |        |                                                             |
| View All Bookings           | Admin  | Monitor every booking across the platform with full details |
| View Platform Analytics     | Admin  | Access aggregate booking volumes, revenue, and city metrics |
| Generate Revenue Report     | Admin  | Export platform-wide financial data for reconciliation      |
| Send Platform Notification  | Admin  | Broadcast alerts to all or targeted user groups             |
| Auto-Cancel Expired Booking | System | Cancel pre-bookings where check-in grace period has elapsed |
| Send Expiry Reminder        | System | Notify driver 15 minutes before booking end time            |

Send Booking Confirmation System Auto-dispatch confirmation after successful booking creation

Notify on Checkout System Send checkout summary and receipt notification to driver



## Services

## 4. Class Diagrams — All

ParkEase follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). The ten class diagrams below detail every service in the platform.



## 4.1 Auth / User-Service

The Auth/User-Service is the security gateway for the entire platform. It manages registration, login, JWT token lifecycle, OAuth2 provider integration (Google/GitHub), profile management, password changes, and account deactivation. The User entity carries a vehiclePlate field as a quick reference for single-vehicle accounts, while the Vehicle-Service stores full vehicle records for multi-vehicle drivers.

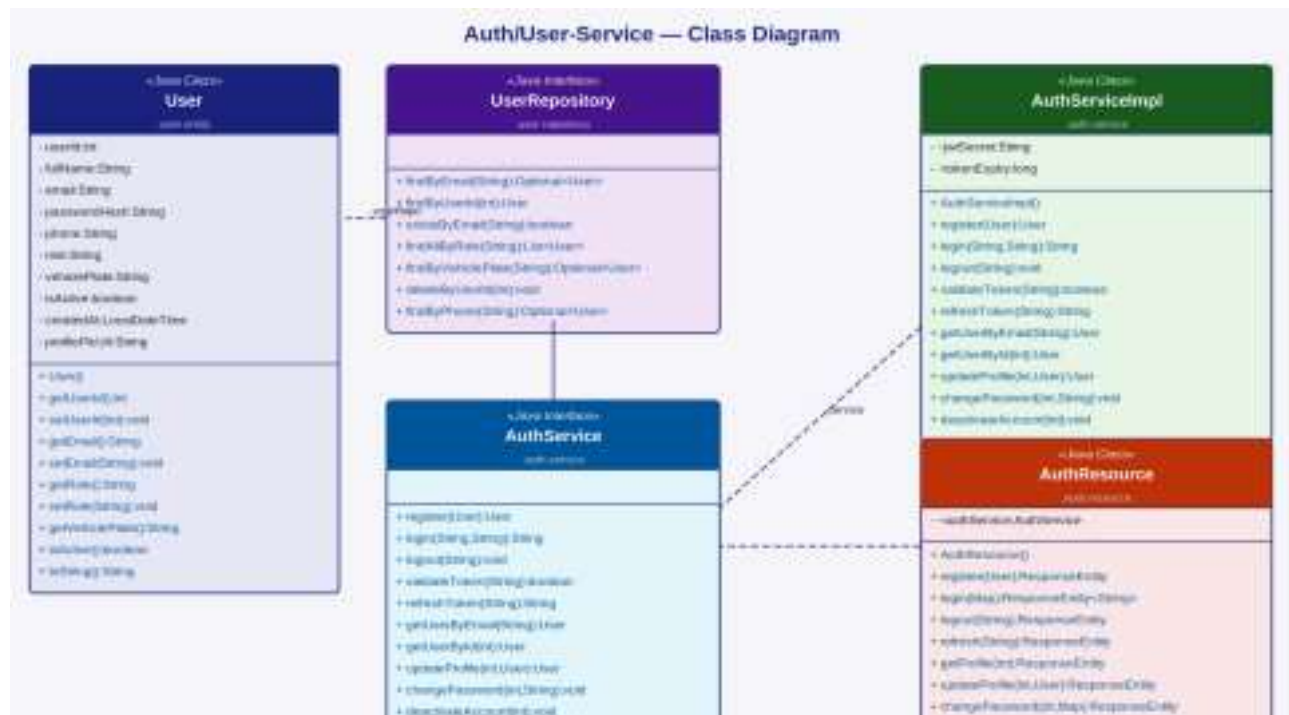


Figure 2: Auth/User-Service — Class Diagram

|                 |                     |                                                                                                                                                                  |
|-----------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 |                     |                                                                                                                                                                  |
| User            | Java Class (Entity) | Stores userId, fullName, email, passwordHash, phone, role (DRIVER/MANAGER/ADMIN), vehiclePlate (quick ref), isActive, createdAt, profilePicUrl                   |
| UserRepository  | Java Interface      | findByEmail(), findByUserId(), existsByEmail(), findAllByRole(), findByVehiclePlate(), findByPhone(), deleteByUserId()                                           |
| AuthServiceImpl | Java Class          | Implements register, login, logout, JWT generation/validation, OAuth handling, profile update, password change, account deactivation                             |
| AuthService     | Java Interface      | Declares register(), login(), logout(), validateToken(), refreshToken(), getUserByEmail(), getUserById(), updateProfile(), changePassword(), deactivateAccount() |

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

|              |                   |                                                                                                                             |
|--------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------|
| AuthResource | Java Class (REST) | Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/deactivate |
|--------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------|

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

## 4.2 ParkingLot-Service

The ParkingLot-Service manages the profiles and operational state of parking facilities. Each lot stores geolocation (latitude/longitude) for GPS-based proximity discovery, operating hours, total and available spot counts, and an admin-approval flag. The availableSpots counter is decremented atomically on booking and incremented on checkout or cancellation. Admin approval is required before a lot becomes searchable by drivers.

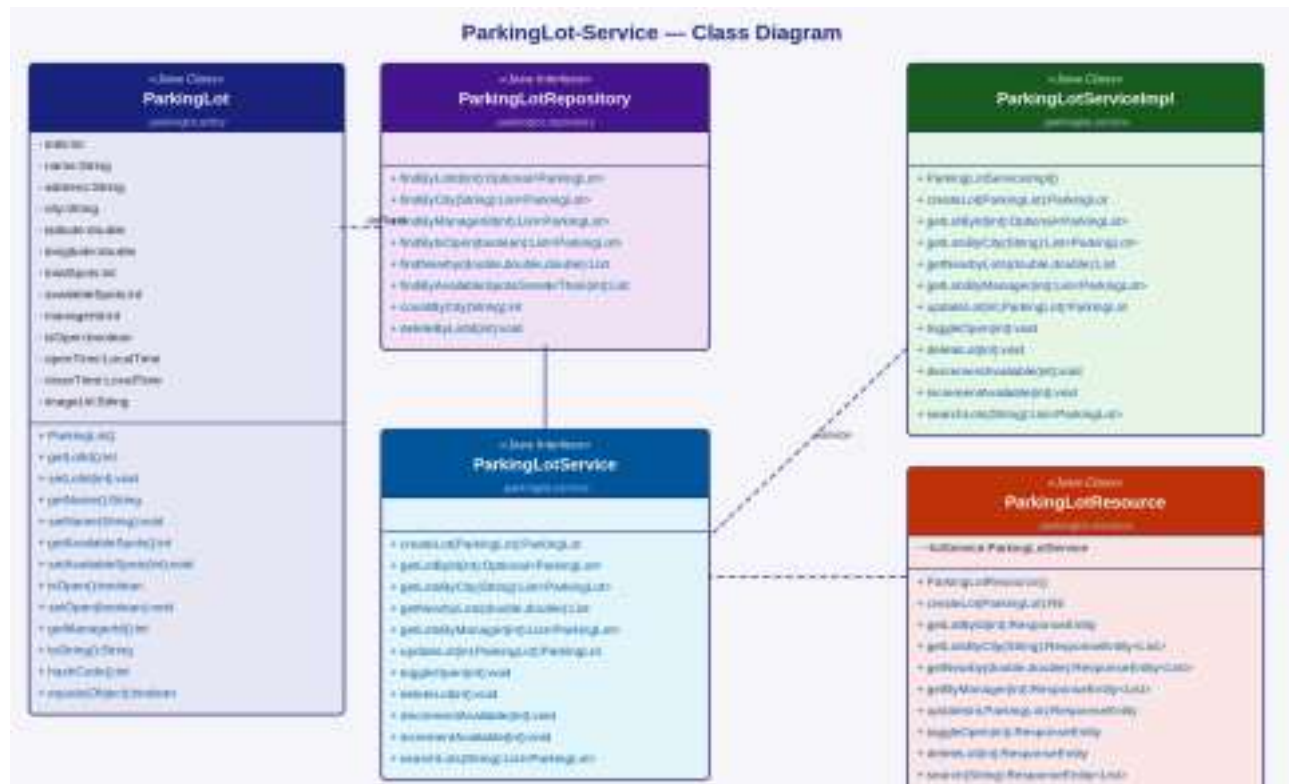


Figure 3: ParkingLot-Service — Class Diagram

|                      |                     |                                                                                                                                                        |
|----------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| ParkingLot           | Java Class (Entity) | lotId, name, address, city, latitude, longitude, totalSpots, availableSpots, managerId, isOpen, openTime, closeTime, imageUrl                          |
| ParkingLotRepository | Java Interface      | findByLotId(), findByCity(), findByManagerId(), findByIsOpen(), findNearby(), findByAvailableSpotsGreaterThanOrEqual(), countByCity(), deleteByLotId() |

|                       |            |                                                                                                                                                                                   |
|-----------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ParkingLotServiceImpl | Java Class | createLot(), getLotById(), getLotsByCity(), getNearbyLots(), getLotsByManager(), updateLot(), toggleOpen(), deleteLot(), decrementAvailable(), incrementAvailable(), searchLots() |
|-----------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

|                    |                   |                                                                                                                 |
|--------------------|-------------------|-----------------------------------------------------------------------------------------------------------------|
|                    |                   |                                                                                                                 |
| ParkingLotService  | Java Interface    | Declares all lot CRUD, geo-proximity search, open-toggle, approval, and available-count management operations   |
| ParkingLotResource | Java Class (REST) | Exposes /lots endpoints: POST (create), GET (by id/city/manager/nearby/search), PUT (update/toggleOpen), DELETE |

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

### 4.3 ParkingSpot-Service

The ParkingSpot-Service manages individual parking spaces within each lot. Spots are typed (Compact, Standard, Large, Motorbike, EV-only) and have vehicle-type compatibility constraints. The status field transitions through Available → Reserved → Occupied → Available. Boolean flags for EV charging and handicapped accessibility enable precise filtering. Bulk spot creation is supported to allow managers to onboard a new lot quickly.



Figure 4: ParkingSpot-Service — Class Diagram

| ParkingSpot     | Java Class (Entity) | spotId, lotId, spotNumber, floor, spotType (COMPACT/STANDARD/LARGE/MOTORBIKE/ EV ), vehicleType (2W/4W/HEAVY), status (AVAILABLE/RESERVED/OCCUPIED), isHandicapped, isEVCharging, pricePerHour |
|-----------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SpotRepository  | Java Interface      | findByLotId(), findByLotIdAndStatus(), findByLotIdAndSpotType(), findByLotIdAndVehicleType(), findBySpotId(), countByLotIdAndStatus(), findByIsEVCharging(), deleteBySpotId()                  |
| SpotServiceImpl | Java Class          | addSpot(), addBulkSpots(), getSpotById(), getSpotsByLot(), getAvailableSpots(),                                                                                                                |

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

|  |  | getByTypeAndLot(), occupySpot(), releaseSpot(), updateSpot(), deleteSpot(), countAvailable() |
|--|--|----------------------------------------------------------------------------------------------|

|              |                   |                                                                                                                           |
|--------------|-------------------|---------------------------------------------------------------------------------------------------------------------------|
| SpotService  | Java Interface    | Declares all spot CRUD, bulk creation, availability query, occupancy state management, and count operations               |
| SpotResource | Java Class (REST) | Exposes /spots endpoints: POST (add/bulk), GET (by lot/id/available/type), PUT (occupy/release/update), DELETE, GET count |

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

## 4.4 Booking / Reservation-Service

The Booking/Reservation-Service is the core orchestration service of the platform. It manages the complete parking booking lifecycle: creation, check-in, check-out, cancellation, and extension. On booking creation it calls the Spot-Service to mark the spot Reserved and the ParkingLot-Service to decrement availableSpots. On check-in it transitions the spot to Occupied. On checkout it computes the final fare and triggers the Payment-Service. On cancellation it releases the spot and triggers a refund where applicable.

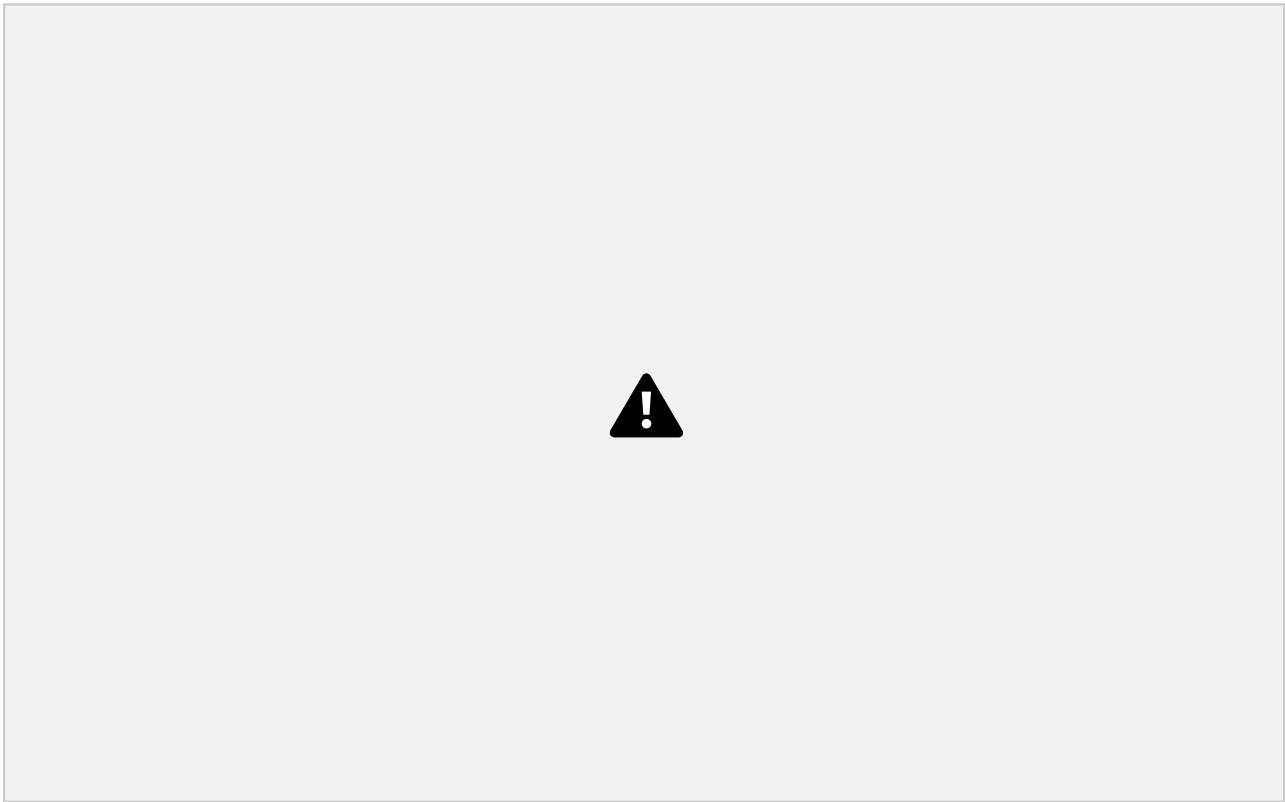


Figure 5: Booking/Reservation-Service — Class Diagram

|         |                     |                                                                                                                                                                                   |
|---------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |                     |                                                                                                                                                                                   |
| Booking | Java Class (Entity) | bookingId, userId, lotId, spotId, vehiclePlate, vehicleType, bookingType (PRE/WALK_IN), startTime, endTime, status (RESERVED/ACTIVE/COMPLETED/CANCELLED ), totalAmount, createdAt |



|                    |                |                                                                                                                                                                      |
|--------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BookingRepository  | Java Interface | findByUserId(), findByLotId(),<br>findBySpotId(), findByStatus(),<br>findByBookingId(),<br>findActiveBySpotId(),<br>findByVehiclePlate(),<br>countByLotIdAndStatus() |
| BookingServiceImpl | Java Class     | createBooking(), getBookingById(),<br>getBookingsByUser(), getBookingsByLot(),                                                                                       |

Confidential | ParkEase Platform | Internal Use Only



#### Parking Lot System | Case Study Document

|                 |                   |                                                                                                                                                     |
|-----------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
|                 |                   |                                                                                                                                                     |
|                 |                   | getActiveBookings(), cancelBooking(),<br>checkIn(), checkOut(), extendBooking(),<br>calculateAmount(), getBookingHistory()                          |
| BookingService  | Java Interface    | Declares all booking CRUD, lifecycle transitions<br>(checkIn/checkOut/cancel/extend), fare<br>calculation, and history retrieval operations         |
| BookingResource | Java Class (REST) | Exposes /bookings endpoints: POST (create),<br>GET (by id/user/lot/active/history), PUT<br>(cancel/checkIn/checkOut/extend), GET<br>calculateAmount |

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

## 4.5 Payment-Service

The Payment-Service processes parking fee payments and manages refunds. Each Payment record is linked to exactly one booking and records the amount, payment mode, transaction ID, currency, and status. The service supports receipt/invoice generation as downloadable PDFs and exposes lot-level revenue aggregation for manager earnings dashboards. Refunds are triggered on eligible booking cancellations and processed back to the original payment mode.



Figure 6: Payment-Service — Class Diagram

|                    |                     |                                                                                                                                                                    |
|--------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    |                     |                                                                                                                                                                    |
| Payment            | Java Class (Entity) | paymentId, bookingId, userId, amount, status (PENDING/PAID/REFUNDED/FAILED), mode (CARD/UPI/WALLET/CASH), transactionId, currency, paidAt, refundedAt, description |
| PaymentRepository  | Java Interface      | findByBookingId(), findByUserId(), findByStatus(), findByTransactionId(), findByPaidAtBetween(), sumAmountByLotId(), countByUserId()                               |
| PaymentServiceImpl | Java Class          | processPayment(), getByBooking(), getByUser(), refundPayment(), getPaymentStatus(), updateStatus(), generateReceipt(), getTotalRevenue(), getTransactionHistory()  |

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

|                |                |                                                                                                                    |
|----------------|----------------|--------------------------------------------------------------------------------------------------------------------|
|                |                |                                                                                                                    |
| PaymentService | Java Interface | Declares all payment processing, refund, receipt generation, revenue aggregation, and history retrieval operations |

|                 |                   |                                                                                                                           |
|-----------------|-------------------|---------------------------------------------------------------------------------------------------------------------------|
| PaymentResource | Java Class (REST) | Exposes /payments endpoints: POST (process), GET (by booking/user/history), POST (refund), GET (status, receipt, revenue) |
|-----------------|-------------------|---------------------------------------------------------------------------------------------------------------------------|

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

## 4.6 Vehicle-Service

The Vehicle-Service manages the fleet of vehicles registered by drivers on the platform. Each Vehicle record stores full details including license plate (unique per owner), make, model, colour, vehicle type (2-Wheeler, 4-Wheeler, Heavy), and an EV flag. The EV flag is cross-referenced by the Booking-Service when assigning EV-charging spots. Drivers can register multiple vehicles and select which vehicle to use at booking time.

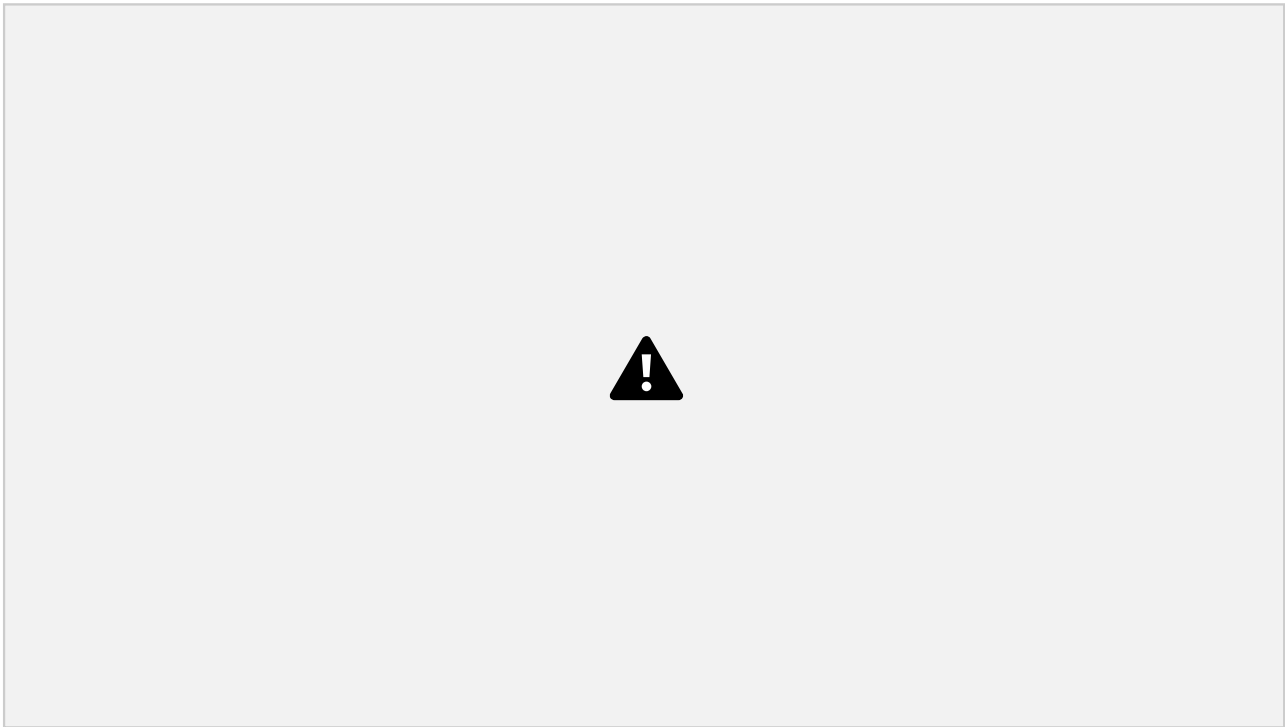


Figure 7: Vehicle-Service — Class Diagram

|                   |                     |                                                                                                                                          |
|-------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------|
|                   |                     |                                                                                                                                          |
| Vehicle           | Java Class (Entity) | vehicleId, ownerId, licensePlate (unique per owner), make, model, color, vehicleType (2W/4W/HEAVY), isEV, registeredAt, isActive         |
| VehicleRepository | Java Interface      | findByOwnerId(), findByLicensePlate(), findByVehicleId(), findByVehicleType(), findByIsEV(), existsByLicensePlate(), deleteByVehicleId() |

|                    |            |                                                                                                                                                                                 |
|--------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VehicleServiceImpl | Java Class | registerVehicle(), getVehicleById(),<br>getVehiclesByOwner(), getByLicensePlate(),<br>updateVehicle(), deleteVehicle(),<br>getVehicleType(), isEVVehicle(),<br>getAllVehicles() |
|--------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

VehicleService Java Interface Declares all vehicle CRUD, plate lookup, type query, and EV-status operations

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

|                 |                   |                                                                                                                         |
|-----------------|-------------------|-------------------------------------------------------------------------------------------------------------------------|
|                 |                   |                                                                                                                         |
| VehicleResource | Java Class (REST) | Exposes /vehicles endpoints: POST (register),<br>GET (by id/owner/plate/all), PUT (update),<br>DELETE, GET (type, isEV) |

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

## 4.7 Notification-Service

The Notification-Service dispatches multi-channel alerts (in-app, email, SMS) for all key parking lifecycle events: booking confirmed, check-in reminder, expiry warning (15 minutes before end time), checkout summary, and payment receipt. It also handles lot manager alerts for new bookings and admin broadcast messages. The service tracks read/unread state and exposes an unread badge count for the UI notification bell.



Figure 8: Notification-Service — Class Diagram

|                         |                     |                                                                                                                                                                    |
|-------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         |                     |                                                                                                                                                                    |
| Notification            | Java Class (Entity) | notificationId, recipientId, type (BOOKING/CHECKIN/EXPIRY/CHECKOUT/PAYMENT/PROMO), title, message, channel (APP/EMAIL/SMS), relatedId, relatedType, isRead, sentAt |
| NotificationRepository  | Java Interface      | findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findByRelatedId(), deleteByNotificationId()                        |
| NotificationServiceImpl | Java Class          | send(), sendBulk(), markAsRead(), markAllRead(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), sendSMS(), getAll()                        |

NotificationService Java Interface Declares notification dispatch (single/bulk/email/SMS), retrieval, read-state management, and deletion operations



|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

|                      |                   |                                                                                                                           |
|----------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------|
| NotificationResource | Java Class (REST) | Exposes /notifications endpoints:<br>getByRecipient, markAsRead, markAllRead,<br>getUnreadCount, delete, sendBulk, getAll |
|----------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------|

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

## 4.8 Analytics / Report-Service

The Analytics/Report-Service computes and exposes occupancy, utilisation, and revenue metrics for parking lots. OccupancyLog records are written at configurable intervals to build a historical dataset for trend analysis. The service computes real-time occupancy rate, hourly breakdown, peak hours, average parking duration, spot type utilisation, and revenue aggregations. A platform-level summary aggregates across all active lots for the admin dashboard.

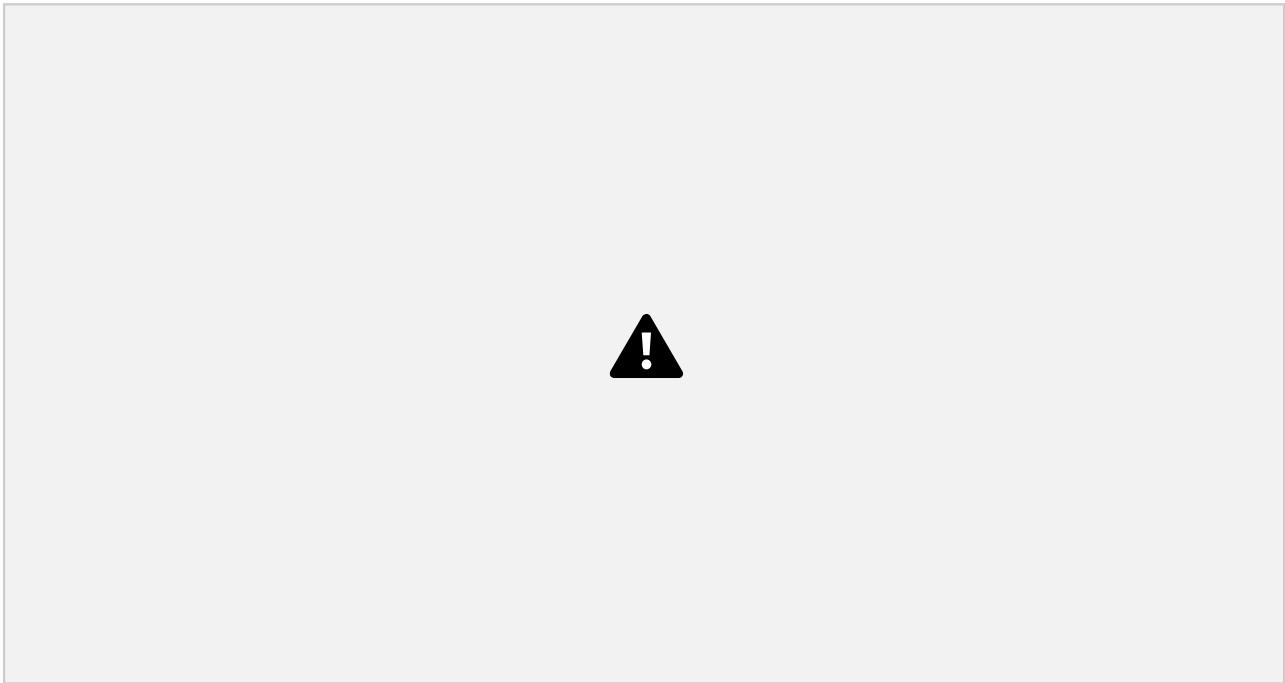


Figure 9: Analytics/Report-Service — Class Diagram

|                      |                     |                                                                                                                                                                                             |
|----------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      |                     |                                                                                                                                                                                             |
| OccupancyLog         | Java Class (Entity) | logId, lotId, spotId, timestamp, occupancyRate, availableSpots, totalSpots, vehicleType — periodic snapshot for trend analysis                                                              |
| AnalyticsRepository  | Java Interface      | findByLotId(),<br>findByLotIdAndTimestampBetween(),<br>avgOccupancyByLotId(),<br>findPeakHoursByLotId(), findByVehicleType(),<br>countByLotIdToday()                                        |
| AnalyticsServiceImpl | Java Class          | logOccupancy(), getOccupancyRate(),<br>getOccupancyByHour(), getPeakHours(),<br>getRevenueByLot(), getRevenueByDay(),<br>getMostUsedSpotTypes(),<br>getAvgDuration(), getPlatformSummary(), |

|                  |                |                                                                                                                             |
|------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------|
|                  |                | generateDailyReport()                                                                                                       |
| AnalyticsService | Java Interface | Declares all occupancy logging, rate computation, peak-hour analysis, revenue aggregation, and report generation operations |

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

|                   |                   |                                                                                                                                                   |
|-------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
|                   |                   |                                                                                                                                                   |
| AnalyticsResource | Java Class (REST) | Exposes /analytics endpoints: GET (occupancyRate, byHour, peakHours, revenue, revenueByDay, spotTypes, avgDuration, platformSummary, dailyReport) |

Confidential | ParkEase Platform | Internal Use Only



**Parking Lot System** | Case Study Document

## 4.9 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC integration layer that wires all underlying microservices together and renders Thymeleaf views for drivers, lot managers, and administrators. It contains three controllers — DriverController, LotManagerController, and AdminController — each injecting relevant service dependencies via Spring @Autowired and delegating to REST Resource endpoints via RestTemplate for inter-service coordination.

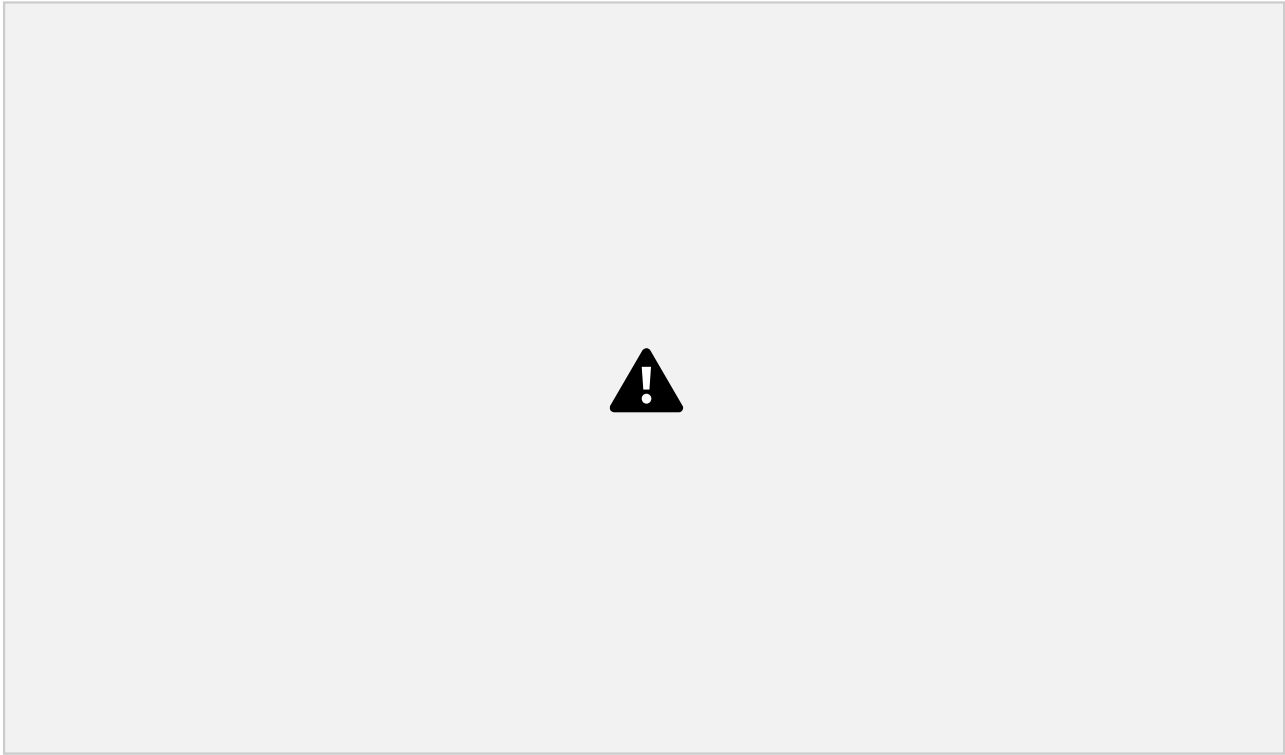


Figure 10: Website-Controller — Class Diagram

|                      |                  |                                                                                                                                                                                                                                                                                                                                       |
|----------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      |                  |                                                                                                                                                                                                                                                                                                                                       |
| DriverController     | Java Class (MVC) | home(), register(), login(), viewProfile(), editProfile(), registerVehicle(), viewMyVehicles(), searchLots(), viewNearbyLots(), viewLotDetail(), viewAvailableSpots(), createBooking(), viewMyBookings(), cancelBooking(), checkIn(), checkOut(), extendBooking(), makePayment(), viewPaymentHistory(), viewNotifications(), logout() |
| LotManagerController | Java Class (MVC) | managerDashboard(), createLot(), editLot(), deleteLot(), toggleOpen(), addSpot(), addBulkSpots(), editSpot(), deleteSpot(), viewAllBookings(), viewActiveBookings(), updateBookingStatus(), viewRevenue(),                                                                                                                            |

Confidential | ParkEase Platform | Internal Use Only



|  |  |                        |
|--|--|------------------------|
|  |  |                        |
|  |  | viewOccupancyReport(), |



|                 |                  |                                                                                                                                                                                                                                                                                            |
|-----------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 |                  | viewPeakHours(), generateDailyReport()                                                                                                                                                                                                                                                     |
| AdminController | Java Class (MVC) | adminDashboard(), manageAllUsers(), suspendUser(), deleteUser(), manageAllLots(), approveLot(), deleteLot(), viewAllBookings(), viewAllPayments(), viewPlatformAnalytics(), viewPlatformRevenue(), sendPlatformNotification(), viewAllVehicles(), generateRevenueReport(), viewAuditLogs() |

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

## 5. Microservices

### Architecture Overview

ParkEase is structured as a set of independently deployable microservices following the same layered pattern as the EShoppingZone reference. Each service owns its own database schema, REST API contract, service interface, and business logic. Synchronous inter-service calls use RestTemplate; asynchronous notification events and occupancy-log writes are routed via RabbitMQ.

|                      |                           |                                                                        |
|----------------------|---------------------------|------------------------------------------------------------------------|
|                      |                           |                                                                        |
| auth-service         | com.parkease.auth         | User registration, login, JWT, OAuth2 (Google/GitHub)                  |
| parkinglot-service   | com.parkease.parkinglot   | Lot profiles, geo-proximity search, approval, open-toggle              |
| spot-service         | com.parkease.spot         | Spot CRUD, type/status management, bulk creation, EV/handicapped flags |
| booking-service      | com.parkease.booking      | Booking lifecycle: create, check-in, check-out, cancel, extend         |
| payment-service      | com.parkease.payment      | Payment processing, refunds, receipt generation, revenue aggregation   |
| vehicle-service      | com.parkease.vehicle      | Vehicle registration, plate lookup, type/EV queries                    |
| notification-service | com.parkease.notification | In-app, email, SMS alerts; bulk dispatch; read-state tracking          |
| analytics-service    | com.parkease.analytics    | Occupancy logs, peak hours, revenue reports, platform summary          |
| parkease-web         | com.parkease.web          | Spring MVC controllers, Thymeleaf view rendering                       |



## Requirements

|                   |                                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|                   |                                                                                                                                                       |
| Performance       | Lot search and spot availability queries must respond within 1.5 seconds for 10,000 concurrent users                                                  |
| Scalability       | Each microservice independently scalable via Docker/Kubernetes; Redis caches available-spot counts for high-frequency reads                           |
| Security          | Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for social login; HTTPS enforced; booking APIs gated by JWT                  |
| Availability      | 99.9% uptime SLA; health-check endpoints per service; graceful degradation if analytics or notification service is temporarily unavailable            |
| Data Integrity    | Spot occupation is atomic using optimistic locking to prevent double-booking of the same spot at the same time                                        |
| Geo-Proximity     | Nearby lot discovery uses the Haversine formula on stored latitude/longitude with a configurable search radius in kilometres                          |
| Automated Cleanup | Expired pre-bookings (check-in grace period elapsed) are auto-cancelled within 5 minutes by a Spring @Scheduled job                                   |
| Audit Trail       | All critical actions (lot approval, booking cancellation, manual checkout, user suspension) are logged with actor, timestamp, and before/after values |
| Usability         | Responsive UI on desktop and mobile; accessible map-based lot search; WCAG 2.1 AA compliance                                                          |
| Maintainability   | Services independently deployable; RESTful API contracts versioned under /api/v1/                                                                     |



## Stack

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
|                   |                                                                 |
| Backend Framework | Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA) |

## 6. Non-Functional

## 7. Proposed Technology

|                      |                                                                                                                       |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|
| Authentication       | JWT (JSON Web Tokens), Spring Security OAuth2, Google / GitHub OAuth                                                  |
| Database             | MySQL (primary relational store per service); Redis (available-spot counter cache, session store)                     |
| Geo-Proximity Search | MySQL spatial queries with Haversine formula; optionally PostGIS for advanced spatial queries                         |
| Map / Location UI    | Leaflet.js or Google Maps JavaScript API for interactive lot map with real-time availability markers                  |
| Frontend             | Thymeleaf templates with Bootstrap 5 and Leaflet.js; or React.js SPA with redux-toolkit                               |
| File / Image Storage | AWS S3 for lot images and PDF receipt storage; CloudFront CDN for delivery                                            |
| Payment Gateway      | Razorpay / Stripe for card and UPI payments; internal wallet module for prepaid balance                               |
| Notifications        | JavaMailSender (SMTP) for email; Twilio or AWS SNS for SMS; WebSocket for in-app real-time alerts                     |
| Messaging            | RabbitMQ for asynchronous notification dispatch and occupancy-log event streaming                                     |
| Scheduled Jobs       | Spring @Scheduled or Quartz Scheduler for expired booking cancellation, expiry reminders, and daily report generation |
| Containerisation     | Docker with Docker Compose; Kubernetes for production autoscaling and blue-green deployments                          |
| API Documentation    | Swagger / OpenAPI 3.0 for all REST endpoints across all services                                                      |
| CI/CD                | GitHub Actions or Jenkins pipeline for automated build, test, and deployment                                          |

Confidential | ParkEase Platform | Internal Use Only



Parking Lot System | Case Study Document

## 8. Glossary

|              |                                                                                                               |
|--------------|---------------------------------------------------------------------------------------------------------------|
|              |                                                                                                               |
| Driver       | A registered vehicle owner who uses ParkEase to find and reserve parking spots                                |
| Lot Manager  | A parking facility operator who registers lots, manages spots, and views earnings on ParkEase                 |
| Parking Lot  | A physical parking facility registered on the platform with a defined spot layout and operating hours         |
| Parking Spot | An individual parking space within a lot, identified by spot number and floor, with type and pricing metadata |

|                  |                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Booking          | A reservation that links a driver, a specific spot, and a time window with a lifecycle status                                    |
| Pre-Booking      | An advance reservation made before arriving at the lot, optionally with an upfront deposit                                       |
| Walk-in Booking  | An immediate booking created upon arrival at the lot without prior reservation                                                   |
| Check-In         | The driver's digital confirmation of arrival at the reserved spot; transitions spot status to Occupied                           |
| Check-Out        | The driver's digital confirmation of departure; triggers fare computation and payment processing                                 |
| Spot Type        | The category of a parking space: Compact, Standard, Large, Motorbike, or EV-only                                                 |
| Vehicle Type     | The class of a vehicle: 2-Wheeler (2W), 4-Wheeler (4W), or Heavy vehicle                                                         |
| EV Charging      | A spot feature indicating the presence of an electric vehicle charging point                                                     |
| Occupancy Rate   | Real-time metric: $(\text{number of occupied spots} / \text{total spots}) \times 100\%$                                          |
| Peak Hours       | The time windows during a day when a parking lot experiences its highest occupancy                                               |
| Geo-proximity    | Location-based lot discovery using the Haversine formula on GPS latitude/longitude coordinates                                   |
| Fare Calculation | Computed as $(\text{checkOutTime} - \text{checkInTime in hours}) \times \text{spot.pricePerHour}$ , with a minimum 1-hour charge |
| Grace Period     | The configurable time window after a pre-booking's start time during which the driver may still check in                         |
| Approval         | The admin action required to activate a newly registered parking lot before it becomes searchable by drivers                     |